M&A OPERATING SYSTEM

Product Design

MAOS Knowledge MCP Server

Draft v1.0 · June 2026

Andrew Bush (www.maoperatingsystem.com/bio-andrew-bush)

Contents

About This Document

This document is part of the **M&A Operating System** product design specification series. It describes the intended design, architecture, and behaviour of the platform within. It is intended for internal distribution across product, engineering, and design review functions.

This is a living specification. It reflects design intent at the time of publication and will be updated as the product evolves.

About M&A Operating System

M&A Operating System is a boutique advisory firm focused on helping organizations modernize the operational foundations required for scalable analytics, trusted enterprise information, and practical AI adoption.

Our work sits at the intersection of enterprise data strategy, information architecture, operating model modernization, and transformation execution. We specialize in helping organizations design practical, business-aligned approaches to enterprise information management that support operational scalability, analytics enablement, governance, and modern AI capabilities.

A core part of our approach is the recognition that successful data and AI initiatives are not driven solely by technology platforms. Long-term success depends on how well organizations structure, model, govern, operationalize, and align information with real business processes and decision-making.

Our Product Design document series reflects this philosophy. These documents are intended to provide practical frameworks, methodologies, architectural concepts, and implementation guidance that help organizations move beyond theoretical strategy into executable operational design.

M&A Operating System's approach combines: - Enterprise data and AI strategy - Subject-based data modeling and information architecture - Semantic and operational data design - Governance and organizational alignment - Operating model modernization - Transformation execution and delivery leadership - Practical AI operationalization and readiness

Our goal is simple: help organizations build practical, scalable, and trusted information foundations that improve operational effectiveness, accelerate analytics and AI adoption, and support better business outcomes.

Find Out More

To learn more about M&A Operating System, explore the platform, or speak with our team: maoperatingsystem.com

01 — Overview

Product: MAOS Knowledge MCP Server
Version: 1.0
Date: 2026-06-16
Protocol: MCP 2025-06-18
Author: Andrew Bush / M&A Operating System

Problem Statement

MAOS platform components — MCP servers providing value-added capabilities, agent pipelines, and downstream agent systems — each carry their own local copies of prompts, skill definitions, agent personas, and reference documentation. There is no single authoritative location to publish, version, and retrieve these assets. When a prompt is updated, every consumer must be updated in lockstep. There is no discovery mechanism and no governed access pattern.

Solution

A centralised MCP server exposing a structured knowledge directory over the Model Context Protocol. Every MAOS component and any authorised MCP client connects to this server to discover and consume resources, prompts, skills, commands, and agent definitions from one place. The server is the single source of truth for all reusable AI assets across the platform.

The design assumes that all content can be aligned to a knowledge hierarchy that mirrors the organisation itself — structured around the organisation's hierarchy, its existing business processes, and the applications and services that support them. This means the directory tree naturally reflects how the business is structured: domains map to business units or functions, sub-domains to teams or process areas, and application nodes to the specific systems and services in use. AI assets are authored and discovered in the same context as the work they support.

Background: MCP Primitives

The Model Context Protocol defines four core primitives that servers use to expose capabilities to clients. In a typical MCP deployment each server manages its own primitives locally — prompts, skills, and agent definitions live alongside the code that uses them, with no shared authoring location or distribution mechanism.

The Knowledge MCP Server changes this. It acts as a **central repository for all MCP primitives across the ecosystem** — a single, Git-versioned location where primitives are authored, reviewed, and published once,

then consumed by any authorised MCP client, agent, or server on the network. When a prompt or skill is updated, every consumer receives the change automatically through MCP notifications. No local copies, no lockstep updates, no drift.

Primitive	What it is	How the Knowledge Server centralises it
Resource	Named content items a server exposes for clients to discover (`resources/list`) and read (`resources/read`). Identified by a URI. Can be files, database records, or any structured data.	Every reference document, schema, and configuration file in the knowledge directory is versioned in Git and surfaced as a resource addressable by its `file:///knowledge/...` URI — one authoritative copy available to all consumers.
Prompt	Parameterised message templates the server registers (`prompts/list`) and renders on demand (`prompts/get`). Arguments are substituted at render time; the result is a `messages[]` array ready for direct LLM submission.	Prompt templates, skill entry points, command definitions, and agent system prompts are all authored in one place, version-controlled, and distributed on demand. Any MCP client calls `prompts/get` and receives a fully rendered message array — no local template management required.
Tool	Callable functions the server exposes for clients and models to invoke. Each tool has a typed input schema; the server executes the function and returns `content` (display text) and `structuredContent` (typed payload).	Ten typed tools give the wider ecosystem structured, type-safe access to every content category — skills, commands, agents, prompts, and resources — without each consumer needing to implement its own parsing or rendering logic.
Notification	Server-to-client push messages signalling state changes. Clients subscribe to specific resource URIs or to list-change events and receive notifications when content is added, removed, or updated.	When any file in the knowledge directory changes, subscribed clients across the entire ecosystem are notified instantly — eliminating polling and ensuring every agent and server is always working from the latest published version.

Design Principles

P1 — Single source of truth. One server, one directory. No mirrors, no local copies that diverge.

P2 — Filesystem native. The knowledge directory is a plain filesystem tree versioned in Git. Any developer can read, write, and deploy content with standard tools. No proprietary database required to author.

P3 — MCP protocol-first. All access is through the MCP 2025-06-18 specification. No bespoke APIs, no direct filesystem mounts for consumers.

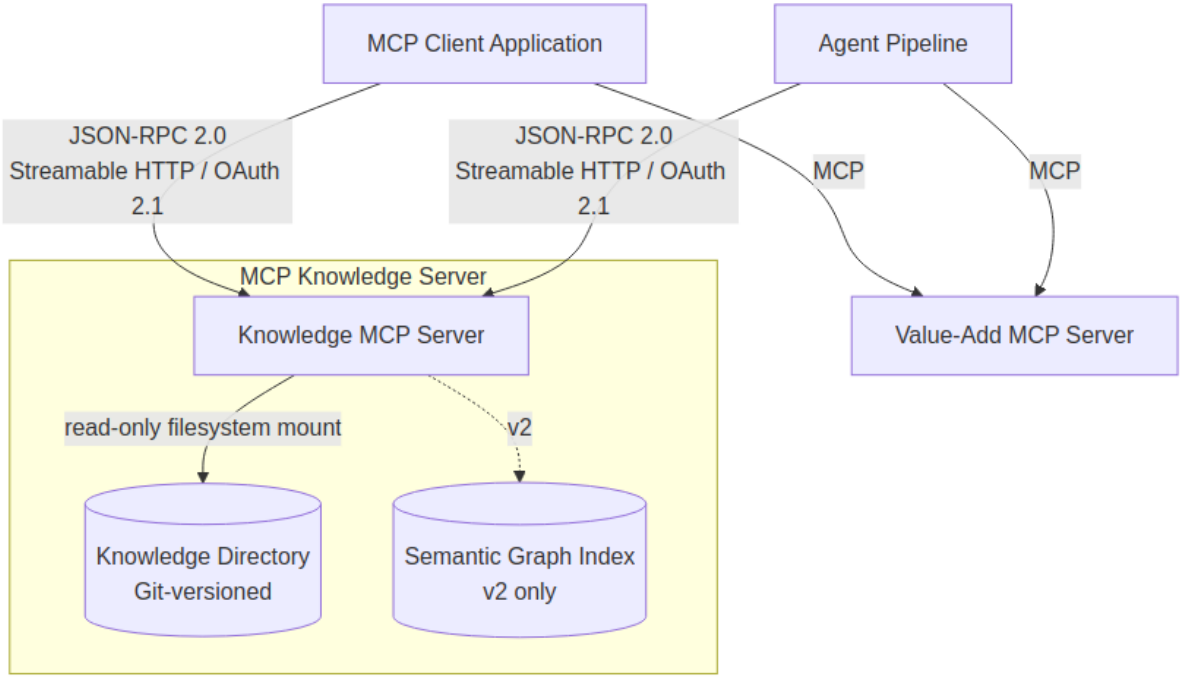
P4 — Type-safe content. Five distinct content types — resources, prompts, skills, commands, agents — each with a defined schema, discovery path, and rendering contract.

P5 — Folder URI consistency. Passing a folder URI to any list, read, or get operation returns all entries scoped to that folder. This behaviour is uniform across all content types and all tools.

P6 — Read-only enforcement. The server never writes to the knowledge directory. All mutation is performed by authorised humans or deployment pipelines operating through version control.

P7 — OAuth 2.1 on all remote connections. Every token is bound to this specific server via RFC 8707. No token sharing across servers.

System Context



Protocol and Capability Declaration

Transport: Streamable HTTP (HTTPS + SSE). Message format: JSON-RPC 2.0. Auth: OAuth 2.1 + PKCE with RFC 8707 resource binding.

```
{
  "protocolVersion": "2025-06-18",
  "capabilities": {
    "resources": { "subscribe": true, "listChanged": true },
    "prompts":   { "listChanged": true },
    "tools":     { "listChanged": false }
  },
  "serverInfo": {
    "name":    "maos-knowledge-mcp-server",
    "title":   "MAOS Knowledge MCP Server",
    "version": "1.0.0"
  },
  "instructions": "Serves the MAOS knowledge directory as resources, prompts, skills, commands,
and agent definitions. All content is under file:///knowledge/. Pass a folder URI to any list or
get operation to enumerate entries. Use typed tools for skills, commands, and agents."
}
```

Technology Stack

Component	Technology
Language	Python 3.12+
MCP SDK	`mcp[cli]` (Python), official SDK
ASGI server	Uvicorn
Auth	OAuth 2.1 + PKCE, `python-jose`, `authlib`
Full-text search	SQLite FTS5 (embedded)
Filesystem watch	`watchfiles`
Front-matter parsing	`python-frontmatter`
Deployment	Docker / OCI, read-only volume mount

Consumers at Launch

- **AI Chat Platform** — the primary conversational AI front end; registers this server in its MCP tool registry as **MCP Knowledge** to provide end users with access to skills, guidance documents, and prompt templates during conversations
- Agent pipelines — orchestrators and sub-agents performing multi-step workflows
- Value-add MCP servers — servers that enrich their own capabilities with knowledge assets
- Claude Code CLI sessions operating in MAOS context

Decisions Log

ID	Decision
D-001	Protocol locked to `2025-06-18` for v1.0
D-002	Python 3.12 + FastMCP as server stack
D-003	Knowledge directory is Git-native, read-only mount
D-004	SQLite FTS5 for search — no external search dependency in v1.0
D-005	Five typed sub-folders are mandatory at every application leaf
D-006	`get_skill` returns `rendered_prompt` and `get_command` returns `resolved_command` in their `structuredContent` — no separate invoke tools are required
D-007	Prompt names in `prompts/list` are file URIs — the same URI returned by `resources/list` for the same file. Uniqueness is guaranteed by the filesystem.

Open Decisions

OD-001 — Search index persistence (MEDIUM): Rebuild on startup is simple and always correct; acceptable if startup time is under 10 seconds. Recommendation: rebuild for v1.0, revisit if startup latency is a problem in practice.

OD-002 — Write path for AI-generated content (MEDIUM): Agents may eventually need to propose new knowledge content. Recommendation: implement via GitHub PR tool when a confirmed requirement exists — not in v1.0.

OD-003 — Content entitlements (MEDIUM): The current design assumes that all knowledge held within this service is non-sensitive and generally accessible to any authenticated client — even if a given piece of content is not relevant or useful to every consumer. No per-content or per-subtree access controls are implemented. If future requirements introduce sensitive content (e.g. restricted agent definitions, confidential process documentation, or proprietary prompt IP), a claims-based entitlement model scoped to directory sub-trees will be needed. Recommendation: proceed with open access for v1.0; revisit when a confirmed requirement for content-level access control exists.

OD-004 — Embedding model versioning (MEDIUM, v2): The pgvector index is tied to a specific embedding model and dimension. Changing the model requires a full re-index. Store `embedding_model` and `embedding_dimensions` as metadata on every pgvector row from day one of v2 deployment so the server can detect stale embeddings after an upgrade.

02 — Knowledge Directory and Content Types

Product: MAOS Knowledge MCP Server
Version: 1.0
Date: 2026-06-16

Directory Layout

The knowledge directory is a Git repository mounted read-only into the server process. The tree is arbitrarily deep above the application node. The five typed sub-folders are mandatory at every application leaf.

```
knowledge/
└── {domain}/
    └── {sub-domain}/                    # arbitrary depth
        └── {application-mcp-server}/   # one per application
            ├── resources/               # raw reference files
            │   ├── data-model.md
            │   ├── glossary.json
            │   └── schemas/
            │       └── concept-entity.json
            ├── prompts/                 # parameterised LLM templates
            │   ├── maturity-assessment.prompt.md
            │   └── onboarding-survey.prompt.md
            ├── skills/                  # multi-file workflow packages
            │   ├── gmail-triage/
            │   │   ├── SKILL.md         # entry point — required
            │   │   └── parse_state.py
            │   └── data-lineage-review/
            │       └── SKILL.md
            ├── commands/                # discrete executable definitions
            │   ├── run-quality-check.cmd.json
            │   └── generate-report.cmd.md
            └── agents/                 # agent persona definitions
                ├── dda-analyst.agent.md
                └── pipeline-orchestrator.agent.json
```

All path segments use kebab-case. No spaces, no underscores, no uppercase. The application segment must match the name in the application's own MCP server manifest.

File Suffix Registry

Folder	Recognised Suffixes	Content Type
`resources/`	`*` (any)	Resource
`prompts/`	`.prompt.md`, `.prompt.json`	Prompt

Folder	Recognised Suffixes	Content Type
`skills/`	`SKILL.md` (one per skill sub-directory)	Skill entry point
`commands/`	`.cmd.md`, `.cmd.json`	Command
`agents/`	`.agent.md`, `.agent.json`	Agent

Files not matching a typed suffix within a typed folder are surfaced as raw resources but excluded from typed registries.

URI Scheme

All content is addressed using `file://` URIs rooted at `file:///knowledge/`. The server enforces this prefix on every incoming URI.

```
file:///knowledge/{domain}/{sub-domain}/{app}/{kind}/{+path}
```

Folder URI behaviour: Passing a trailing-slash URI to any list, read, or get operation returns all direct children scoped to that folder. This is uniform across all content types and all MCP methods. Directory entries are identified with MIME type `inode/directory`.

Example URIs:

```
# Specific file
file:///knowledge/maos/dda/data-design-authority/resources/glossary.json

# Folder listing
file:///knowledge/maos/dda/data-design-authority/skills/

# Skill entry point
file:///knowledge/maos/dda/data-design-authority/skills/gmail-triage/SKILL.md

# Application root
file:///knowledge/maos/dda/data-design-authority/
```

> **Note:** While all content types are reachable via the generic URI scheme above, each typed content category also has its own dedicated entry points. Skills, prompts, commands, and agents can be discovered and retrieved through their respective typed tools (`get_skill`, `get_prompt`, `list_prompts`, `get_command`, `list_agents`, `load_agent`) and via the `prompts/*` MCP primitives. These typed entry points return structured metadata, rendered templates, and resolved definitions — not just raw file content.

Content Type Schemas

Resource

Any file in `resources/`. No structural convention. Front-matter optional and treated as metadata only. Returns `text` (UTF-8) or `blob` (base64) via `resources/read`. Directory URIs return a listing of children.

Prompt (`.prompt.md` / `.prompt.json`)

Parameterised templates rendered into a `messages[]` array for direct LLM submission.

Front-matter schema:

```
---
name:        maturity-assessment       # kebab-case, unique within app
title:       Data Maturity Assessment
version:     1.1.0
description: Guides an organisation through a structured data maturity survey
tags:
  - governance
  - assessment
arguments:
  - name:        org_name
    description: Organisation name
    required:    true
  - name:        scope
    description: Assessment domain
    required:    false
    default:     "General"
---
You are conducting a data maturity assessment for {{org_name}}.
Scope: {{scope}}...
```

Prompt name convention: The `name` field in `prompts/list` responses is the file URI — the same URI returned by `resources/list` for the same file. This eliminates a second identifier for the same resource. `prompts/get` and all typed get tools accept the URI directly. Uniqueness is guaranteed by the filesystem.

Rendering contract: `prompts/get` and `get_prompt` substitute all `{{argument}}` references and return a `messages[]` array with a single `user` role message.

Skill (`{skill-name}/SKILL.md`)

Multi-file packages encoding a reusable, multi-step workflow. `SKILL.md` at the root of the skill sub-directory is the entry point. Support files in the same directory are accessible individually as resources.

Front-matter schema:

```
---
skill:       gmail-triage             # matches sub-directory name
title:       Gmail Triage Skill
version:     1.2.0
description: Runs a Gmail triage and todo-management cycle
tags:
  - email
  - productivity
triggers:
  - "run email triage"
  - "check my inbox for todos"
inputs:
  - name:     max_threads
    type:     integer
    default:  50
    description: Maximum threads to process per cycle
outputs:
  - action_list
  - reply_needed
dependencies:
  - gmail-mcp
  - google-drive-mcp
files:
  - path:  parse_state.py
    role:  support
    description: Parses triage state from Drive markdown
---
# Gmail Triage Skill
...step-by-step workflow instructions...
```

Rendering contract: `get_skill` returns the rendered `SKILL.md` body with `{{input_name}}` references substituted in the `rendered_prompt` field of `structuredContent`, alongside full typed metadata. No separate invocation tool exists.

Command (`.cmd.json` / `.cmd.md`)

Discrete, single-invocation executable definitions carrying a `command` string and a `danger_level` declaration.

JSON schema:

```
{
  "name":        "run-quality-check",
  "title":       "Run Data Quality Check",
  "version":     "1.0.0",
  "description": "Executes a quality gate check against a target entity",
  "tags":        ["governance", "quality"],
  "command":     "quality-check --entity {{entity_id}} --profile {{profile}}",
  "arguments": [
    { "name": "entity_id", "type": "string",  "required": true  },
    { "name": "profile",   "type": "string",  "required": false, "default": "standard" }
  ],
  "returns":      "quality_report",
  "danger_level": "read-only",
  "target_tool":  "target-mcp-server"
}
```

Danger levels:

Level	Meaning	Client Behaviour
`read-only`	No state mutation	May invoke without confirmation
`write`	Creates or modifies state	Should present confirmation prompt
`destructive`	Irreversibly modifies or deletes	Must require explicit confirmation

Rendering contract: `get_command` substitutes supplied arguments into the `command` string and returns the result in the `resolved_command` field of `structuredContent`. It does not execute. No separate invocation tool exists.

Agent (`.agent.md` / `.agent.json`)

Persistent specifications of autonomous actors — persona, model, tool allowlist, memory configuration, associated skills.

Front-matter schema:

```
---
agent:       dda-analyst
title:       DDA Data Analyst Agent
version:     2.0.0
description: Chief Data Officer's second brain
tags:
  - analyst
  - governance
model:       claude-sonnet-4-6
temperature: 0.2
tools_allowed:
  - get_resource
  - search_knowledge
  - get_skill
  - get_command
memory:
  type:      supabase-pgvector
  namespace: dda-analyst
  ttl_days:  90
skills:
  - data-lineage-review
  - gmail-triage
system_prompt_extends: base-cdaio-persona  # optional — prepends base agent system prompt
---
You are the DDA Analyst — the Chief Data Officer's second brain...
```

Rendering contract: `load_agent` returns the full agent metadata in `structuredContent` plus a `messages[]` array containing the system prompt as a `system` role message. If `system_prompt_extends` is set, the base agent's system prompt is prepended.

Content Type to MCP Primitive Mapping

Content Type	`resources/list`	`resources/read`	`prompts/list`	`prompts/get`	Tools
Resource		raw content	—	—	`get_resource`, `search_resources`
Prompt	raw file	raw content	with arguments	rendered `messages[]`	`get_prompt`, `list_prompts`
Skill	all files	any file	SKILL.md as template	rendered SKILL.md	`get_skill`
Command	raw file	raw content	as instruction	rendered instruction	`get_command`
Agent	raw file	raw content	system prompt	rendered system prompt	`load_agent`, `list_agents`

Version Control and Deployment

1. Author creates or edits files in a feature branch
2. PR reviewed and merged to main
3. CD pipeline syncs main to the mounted filesystem
4. Server detects change via `watchfiles`, rebuilds content index, emits `notifications/resources/list_changed` and `notifications/prompts/list_changed` to subscribed clients
5. Clients re-call `resources/list` or `prompts/list` to refresh

The server never writes to the knowledge directory under any code path.

03 — Server Surface

Product: MAOS Knowledge MCP Server
Version: 1.0
Date: 2026-06-16

MCP Primitives

Resources

All five content types are surfaced as raw file content through the resource primitive. The host application or model determines what to do with the content.

`resources/list` — paginated list of all files in the knowledge directory.

```
// Response (one entry shown)
{
  "resources": [{
    "uri":          "file:///knowledge/maos/dda/data-design-authority/resources/get_started.md",
    "name":         "get_started.md",
    "title":        "DDA Get Started & Onboarding Guide",
    "mimeType":     "text/md",
    "size":         8192,
    "annotations":  { "audience": ["user", "assistant"], "priority": 0.8,
                      "lastModified": "2026-05-01T09:00:00Z" }
  }],
  "nextCursor": "next-page-cursor"
}
```

`resources/templates/list` — URI templates for parameterised access.

```
{ "resourceTemplates": [
    { "uriTemplate": "file:///knowledge/{+path}",
      "name": "knowledge-any",
      "title": "Knowledge Directory — Any Path",
      "description": "Resolves any path. Trailing slash returns directory listing." },
    { "uriTemplate": "file:///knowledge/{domain}/{subdomain}/{app}/skills/{skill}/SKILL.md",
      "name": "knowledge-skill", "title": "Knowledge Skill Entry Point", "mimeType": "text/
markdown" },
    { "uriTemplate": "file:///knowledge/{domain}/{subdomain}/{app}/agents/{+path}",
      "name": "knowledge-agent", "title": "Knowledge Agent Definition" }
]}
```

`resources/read` — retrieve a file or directory listing.

```
// File response
{ "contents": [{ "uri": "file:///knowledge/.../glossary.json",
                 "mimeType": "application/json", "text": "{ ... }" }] }

// Folder response (trailing-slash URI)
{ "contents": [
    { "uri": "file:///knowledge/.../skills/gmail-triage/",         "mimeType": "inode/
directory" },
    { "uri": "file:///knowledge/.../skills/data-lineage-review/",  "mimeType": "inode/directory" }
]}
```

`resources/subscribe` — subscribe to change notifications on a specific URI. Server emits `notifications/resources/updated` when the file changes on disk.

Prompts

Surfaces prompt, skill, command, and agent files as parameterised templates rendered into `messages[]` arrays.

Prompt name convention: The `name` field in every `prompts/list` entry is the file URI — identical to the URI returned by `resources/list` for the same file. This is the primary identifier across all MCP methods. `prompts/get` and all typed get tools accept the URI directly.

`prompts/list` — all registered templates with their arguments arrays.

```
{ "prompts": [
    { "name": "file:///knowledge/maos/dda/data-design-authority/prompts/maturity-
assessment.prompt.md",
      "title": "Data Maturity Assessment",
      "arguments": [
        { "name": "org_name", "description": "Organisation name", "required": true },
        { "name": "scope",    "description": "Assessment domain",  "required": false }
      ]},
    { "name": "file:///knowledge/maos/dda/data-design-authority/skills/gmail-triage/SKILL.md",
      "title": "Gmail Triage Skill",
      "arguments": [{ "name": "max_threads", "required": false }] },
    { "name": "file:///knowledge/maos/dda/data-design-authority/agents/dda-analyst.agent.md",
      "title": "DDA Data Analyst Agent — System Prompt", "arguments": [] }
]}
```

`prompts/get` — render a template with arguments substituted.

```
// Request
{ "method": "prompts/get",
  "params": { "name": "file:///knowledge/maos/dda/data-design-authority/prompts/maturity-
assessment.prompt.md",
              "arguments": { "org_name": "Acme Corp", "scope": "Data Governance" } } }

// Response
{ "result": { "description": "Data Maturity Assessment for Acme Corp — Data Governance scope",
              "messages": [{ "role": "user",
                             "content": { "type": "text", "text": "You are conducting..." }}] } }
```

Notifications

Notification	Trigger
`notifications/resources/list_changed`	File added or deleted anywhere in the knowledge directory
`notifications/resources/updated`	File content changed on a subscribed URI
`notifications/prompts/list_changed`	Prompt, skill, command, or agent file added, deleted, or renamed

---

Tool Surface

All tools return a `content` array (text summary for display) and a `structuredContent` object (full typed payload for programmatic use).

Content Type	Search (Discovery)	List (Enumeration)	Read
All types	`search_knowledge` (v1), `search`* (v2)	—	—
Resource	`search_resources`	—	`get_resource` (+ `query` for chunks*)
Prompt	`search_prompts`	`list_prompts`	`get_prompt`
Skill	`search_skills`	`get_skill` (folder URI)	`get_skill`
Command	`search_commands`	`get_command` (folder URI)	`get_command`
Agent	`search_agents`	`list_agents`	`load_agent`

*v2 — requires pgvector infrastructure. Typed shortcuts delegate to `search` in v2 deployments with no API change for callers.

Resource Tools

`get_resource` — file content or directory listing by URI.

```
{
  "name": "get_resource",
  "description": "Returns the content of a knowledge resource by URI. Pass a trailing-slash folder
URI to list all entries within that folder. Works for all content types.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "uri": { "type": "string", "description": "file:// URI. Must begin with file:///
knowledge/.",
               "pattern": "^file:///knowledge/" },
      "query": {
        "type": "string",
        "description": "When supplied, activates chunk retrieval — returns the most relevant
sections of the document ranked by similarity to this query rather than the full file content.
Requires v2 infrastructure (pgvector). Ignored for directory URIs and binary files."
      },
      "top_k": {
        "type": "integer",
        "default": 3,
        "minimum": 1,
        "maximum": 10,
        "description": "Number of chunks to return when query is supplied."
      }
    },
    "required": ["uri"]
  }
}
```

```
// Whole file (query omitted)
{ uri, name, mimeType, size, lastModified, isDirectory: false, text | blob }

// Directory (trailing-slash URI)
{ uri, isDirectory: true, entries: [{ uri, name, mimeType }] }

// Chunk retrieval (query supplied, v2)
{ uri, name, mimeType, isDirectory: false,
  chunks: [{ chunk_id, section_heading?, text, similarity_score }] }
```

> When `query` is supplied and `search_enabled` is `False`, returns `-32603` with message `"Chunk retrieval not available — pgvector not configured"`.

> **Chunk-addressed URIs (v2):** Individual chunks are addressable via a fragment identifier on the file URI:
>
> `file:///knowledge/maos/dda/data-design-authority/resources/data-model.md#chunk=3`

> A `resources/read` request or a `get_resource` call on a chunk URI returns only that chunk's text with `mimeType: text/plain`. `search` results may include `resource_link` entries pointing to chunk URIs.

`search_knowledge` — full-text and metadata search across the knowledge directory.

```
{
  "name": "search_knowledge",
  "description": "Full-text and metadata search across the entire knowledge directory using SQLite
FTS5. Searches all content types — resources, prompts, skills, commands, and agents — unless
content_types is supplied to narrow scope. Searches file content, front-matter fields, and
triggers[] arrays. Use typed shortcuts (search_skills, search_agents, etc.) when the content type
is known. In v2, prefer search for better recall.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "Search terms or phrase. Matched against file content, title, description,
and triggers[] arrays."
      },
      "folder_uri": {
        "type": "string",
        "pattern": "^file:///knowledge/",
        "description": "Optional folder URI to restrict search scope."
      },
      "content_types": {
        "type": "array",
        "items": { "type": "string", "enum": ["resource","prompt","skill","command","agent"] },
        "description": "Optional filter. Omit to search all content types."
      },
      "tags": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Filter by tags declared in front-matter. All supplied tags must be present
(AND semantics)."
      },
      "fields": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Front-matter fields to include in each result. E.g. [\"version\",
\"triggers\", \"dependencies\"]. Omit for default snippet-only results."
      },
      "max_results": {
        "type": "integer",
        "default": 10,
        "minimum": 1,
        "maximum": 50
      }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, total_hits, results: [{ uri, name, title, content_type, mimeType, snippet, score, tags?, fields? }] }`

Typed Search Shortcuts

Five convenience wrappers around `search_knowledge` with `content_types` pre-set. These are thin wrappers — no independent index logic.

`search_resources` — scoped to `content_types: ["resource"]`

```
{
  "name": "search_resources",
  "description": "Search the resources/ folder across the knowledge directory. Equivalent to
search_knowledge with content_types set to resource.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query":       { "type": "string" },
      "folder_uri":  { "type": "string", "pattern": "^file:///knowledge/" },
      "tags":        { "type": "array", "items": { "type": "string" } },
      "max_results": { "type": "integer", "default": 10, "minimum": 1, "maximum": 50 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, total_hits, results: [{ uri, name, mimeType, snippet, score }] }`

`search_prompts` — scoped to `content_types: ["prompt"]`

```
{
  "name": "search_prompts",
  "description": "Search prompt templates across the knowledge directory. Equivalent to
search_knowledge with content_types set to prompt. Returns arguments array in results.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query":       { "type": "string" },
      "folder_uri":  { "type": "string", "pattern": "^file:///knowledge/" },
      "tags":        { "type": "array", "items": { "type": "string" } },
      "max_results": { "type": "integer", "default": 10, "minimum": 1, "maximum": 50 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, total_hits, results: [{ uri, name, title, arguments, snippet, score }] }`

`search_skills` — scoped to `content_types: ["skill"]`

```
{
  "name": "search_skills",
  "description": "Search skill definitions across the knowledge directory. Equivalent to
search_knowledge with content_types set to skill. Searches triggers[] arrays — phrase queries like
'run email triage' will surface matching skills. Returns triggers, dependencies, and inputs in
results.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query":       { "type": "string" },
      "folder_uri":  { "type": "string", "pattern": "^file:///knowledge/" },
      "tags":        { "type": "array", "items": { "type": "string" } },
      "max_results": { "type": "integer", "default": 10, "minimum": 1, "maximum": 50 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, total_hits, results: [{ uri, name, title, triggers, dependencies, inputs, snippet, score }] }`

`search_commands` — scoped to `content_types: ["command"]`

```
{
  "name": "search_commands",
  "description": "Search command definitions across the knowledge directory. Equivalent to
search_knowledge with content_types set to command. Returns danger_level and target_tool in
results.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query":       { "type": "string" },
      "folder_uri":  { "type": "string", "pattern": "^file:///knowledge/" },
      "tags":        { "type": "array", "items": { "type": "string" } },
      "max_results": { "type": "integer", "default": 10, "minimum": 1, "maximum": 50 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`:
`{ query, total_hits, results: [{ uri, name, title, danger_level, target_tool, snippet, score }] }`

`search_agents` — scoped to `content_types: ["agent"]`

```
{
  "name": "search_agents",
  "description": "Search agent definitions across the knowledge directory. Equivalent to
search_knowledge with content_types set to agent. Returns model, tools_allowed, and skills in
results.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query":       { "type": "string" },
      "folder_uri":  { "type": "string", "pattern": "^file:///knowledge/" },
      "tags":        { "type": "array", "items": { "type": "string" } },
      "max_results": { "type": "integer", "default": 10, "minimum": 1, "maximum": 50 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, total_hits, results: [{ uri, name, title, model, tools_allowed, skills, snippet, score }] }`

---

v2 Tool

`search` *(v2 — requires pgvector infrastructure)*

```
{
  "name": "search",
  "description": "v2 replacement for search_knowledge. Combines FTS5 keyword search and pgvector
semantic search using Reciprocal Rank Fusion. Accepts identical parameters to search_knowledge —
all typed shortcuts (search_skills, search_agents, etc.) delegate to search in v2 deployments via
content_types. Use search_knowledge when pgvector is unavailable. Requires v2 infrastructure
(pgvector).",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "folder_uri": { "type": "string", "pattern": "^file:///knowledge/" },
      "content_types": {
        "type": "array",
        "items": { "type": "string", "enum": ["resource","prompt","skill","command","agent"] },
        "description": "Optional filter. Omit to search all content types."
      },
      "tags": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Filter by tags. All supplied tags must be present (AND semantics)."
      },
      "top_k": { "type": "integer", "default": 5, "minimum": 1, "maximum": 20 }
    },
    "required": ["query"]
  }
}
```

`structuredContent`: `{ query, results: [{ uri, name, title, content_type, snippet, rrf_score, keyword_rank, semantic_rank }] }`

Prompt Tools

`list_prompts` — all registered prompt templates, optionally scoped.

```
{
  "name": "list_prompts",
  "description": "Returns registered prompt templates. Pass folder_uri to scope to an application.
Includes prompts, skills, commands, and agents as promptable types.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "folder_uri":    { "type": "string", "pattern": "^file:///knowledge/" },
      "content_types": { "type": "array",
                         "items": { "type": "string", "enum":
["prompt","skill","command","agent"] }},
      "cursor":        { "type": "string" }
    }
  }
}
```

`get_prompt` — render a named template with argument substitution.

```
{
  "name": "get_prompt",
  "description": "Renders a named prompt with argument substitution. Returns messages[] ready for
LLM submission. Pass a folder URI as prompt_name to list promptable content under that path.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "prompt_name": { "type": "string",
                       "description": "File URI (e.g. file:///knowledge/maos/dda/data-design-
authority/prompts/maturity-assessment.prompt.md) or folder URI" },
      "arguments":   { "type": "object", "additionalProperties": { "type": "string" } }
    },
    "required": ["prompt_name"]
  }
}
```

`structuredContent`: `{ prompt_name, description, messages: [{ role, content }] }`

Skill Tools

`get_skill` — SKILL.md metadata and rendered content for a named skill. Folder URI lists all skills under that path.

```
{
  "name": "get_skill",
  "description": "Returns the parsed SKILL.md entry point and metadata for a named skill. Pass a
folder URI to list all skills. Set include_files=true to retrieve all files in the skill
package.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "skill_name":    { "type": "string", "description": "Kebab-case skill name or folder URI" },
      "app_uri":       { "type": "string", "pattern": "^file:///knowledge/" },
      "include_files": { "type": "boolean", "default": false }
    },
    "required": ["skill_name"]
  }
}
```

`structuredContent`: `{ kind: "skill", name, title, version, uri, description, triggers, inputs, outputs, dependencies, files: [{ uri, role }], rendered_prompt }`

Command Tools

`get_command` — parsed command definition including command string, arguments, and danger level. Folder URI lists all commands.

```
{
  "name": "get_command",
  "description": "Returns the parsed definition of a named command. Pass a folder URI to list all
commands under that path.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "command_name": { "type": "string", "description": "Kebab-case command name or folder
URI" },
      "app_uri":      { "type": "string", "pattern": "^file:///knowledge/" }
    },
    "required": ["command_name"]
  }
}
```

`structuredContent`: `{ kind: "command", name, title, version, uri, description, command, arguments, returns, danger_level, target_tool, resolved_command, arguments_used }`

> `resolved_command` contains the command string with supplied `arguments` substituted. Pass arguments to `get_command` to receive the resolved string in the same response — no separate tool is required.

Agent Tools

`list_agents` — all registered agent definitions, optionally scoped.

```
{
  "name": "list_agents",
  "description": "Returns all registered agent definitions. Pass folder_uri to scope to an
application.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "folder_uri": { "type": "string", "pattern": "^file:///knowledge/" },
      "cursor":     { "type": "string" }
    }
  }
}
```

`structuredContent`: `{ agents: [{ agent, title, version, description, model, uri, skills, tools_allowed }] }`

`load_agent` — full agent definition with rendered system prompt, ready for runtime instantiation. Folder URI lists agents.

```
{
  "name": "load_agent",
  "description": "Returns the full agent definition including rendered system prompt, model
config, tool allowlist, memory config, and skill list. system_prompt_extends causes the base agent
system prompt to be prepended. Pass a folder URI to list available agents.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "agent_name": { "type": "string", "description": "Kebab-case agent name or folder URI" },
      "app_uri":    { "type": "string", "pattern": "^file:///knowledge/" }
    },
    "required": ["agent_name"]
  }
}
```

`structuredContent`: `{ kind: "agent", agent, title, version, uri, model, temperature, tools_allowed, memory, skills, system_prompt, messages: [{ role: "system", content }] }`

04 — Implementation Reference

Product: MAOS Knowledge MCP Server
Version: 1.0
Date: 2026-06-16

Security Model

Transport

All remote connections use TLS. HTTP is rejected in all environments. The `MCP-Protocol-Version: 2025-06-18` header is required on every HTTP request — missing or unsupported version returns `400 Bad Request`. stdio connections (local, same-machine) rely on OS process isolation.

Authentication

OAuth 2.1 with PKCE (S256 method, mandatory) for all remote clients. Token issuance is delegated to an external authorisation server. Every access token is bound to this server via the RFC 8707 resource indicator:

```
resource=https://knowledge-mcp.maoperatingsystem.com
```

Tokens without a resource indicator, or bound to a different server, are rejected. The server publishes its authorization server metadata at `/.well-known/oauth-authorization-server`.

Path Traversal Prevention

Every incoming URI is validated before any filesystem operation, in order:

1. URI must begin with `file:///knowledge/`
2. URI is URL-decoded and `.` / `..` segments resolved
3. Resolved absolute path must begin with `KNOWLEDGE_ROOT`
4. Any failure at steps 1–3 returns `-32602` immediately with no filesystem access

Scopes

Scope	Access
`knowledge:read`	All content types — default for MAOS platform clients
`knowledge:resources:read`	Resources only
`knowledge:prompts:read`	Prompts only

Scope	Access
`knowledge:skills:read`	Skills only
`knowledge:commands:read`	Commands only
`knowledge:agents:read`	Agent definitions only

Rate Limits

Limit	Default
Requests per minute	120
Search requests per minute	20
Concurrent SSE connections	5
Max response size	10 MB

Exceeded limits return HTTP `429` with a `Retry-After` header.

Error Codes

Code	Meaning
`-32002`	Resource not found
`-32602`	Invalid params — URI prefix violation, path traversal, missing required argument
`-32603`	Internal error — filesystem I/O or search index failure
HTTP `400`	Missing or invalid `MCP-Protocol-Version` header
HTTP `401`	Missing or invalid bearer token
HTTP `403`	Valid token, insufficient scope
HTTP `429`	Rate limit exceeded

Project Structure

```
knowledge-mcp-server/
├── src/
│   └── knowledge_mcp/
│       ├── server.py           # FastMCP entry point
│       ├── config.py           # Pydantic settings, env vars
│       ├── errors.py           # JSON-RPC error constants
│       ├── auth/
│       │   ├── middleware.py   # OAuth 2.1 + PKCE validation
│       │   └── scopes.py       # Scope enforcement
│       ├── registry/
│       │   ├── scanner.py          # Filesystem scanner, content index
│       │   ├── watcher.py          # watchfiles integration, notifications
│       │   ├── search.py           # SQLite FTS5 index manager
│       │   ├── embeddings.py       # v2 — embedding generation, pgvector upsert
│       │   └── chunker.py          # v2 — section/paragraph splitting strategy
│       ├── primitives/
│       │   ├── resources.py    # resources/* handlers
│       │   └── prompts.py      # prompts/* handlers
│       ├── tools/
│       │   ├── resource_tools.py   # extend get_resource with query/top_k (v2)
│       │   ├── prompt_tools.py
│       │   ├── skill_tools.py
│       │   ├── command_tools.py
│       │   ├── agent_tools.py
│       │   ├── search_tools.py     # search_knowledge + five typed shortcuts
│       │   └── search_v2.py        # v2 — search
│       └── content/
│           ├── types.py        # ContentKind enum, ContentEntry dataclass
│           ├── resolver.py     # URI validation and path resolution
│           ├── renderer.py     # Front-matter parsing, template substitution
│           └── mime.py         # MIME detection and allowlist
├── tests/
│   ├── unit/
│   └── integration/
├── Dockerfile
├── docker-compose.yml
└── pyproject.toml
```

Key Implementation Modules

Configuration

```
# src/knowledge_mcp/config.py
from pydantic_settings import BaseSettings
from pathlib import Path

class Settings(BaseSettings):
    knowledge_root:          Path
    server_name:             str  = "maos-knowledge-mcp-server"
    server_version:          str  = "1.0.0"
    host:                    str  = "0.0.0.0"
    port:                    int  = 8080
    oauth_issuer:            str
    oauth_audience:          str   # Must match RFC 8707 resource indicator
    oauth_jwks_uri:          str
    fts_db_path:             Path = Path("/tmp/knowledge-fts.db")
    rate_limit_rpm:          int  = 120
    rate_limit_search_rpm:   int  = 20
    rate_limit_sse_connections: int = 5

    # v2 — pgvector / hybrid search
    supabase_url:               Optional[str] = None
    supabase_service_key:       Optional[str] = None
    embedding_model:            str           = "text-embedding-3-small"
    embedding_dimensions:       int           = 1536
    chunk_size_tokens:          int           = 512
    chunk_overlap_tokens:       int           = 64
    search_enabled:             bool          = False  # set True when pgvector is provisioned

    class Config:
        env_prefix = "KNOWLEDGE_MCP_"
```

> `search` and chunk retrieval via `get_resource` both return `-32603` with message `"Not available — pgvector not configured"` when `search_enabled` is `False`. `search_knowledge` and all typed shortcuts remain operational via FTS5 regardless of this flag.

Content Types

```
# src/knowledge_mcp/content/types.py
from enum import StrEnum
from dataclasses import dataclass, field
from pathlib import Path
from datetime import datetime
from typing import Optional

class ContentKind(StrEnum):
    RESOURCE = "resource"
    PROMPT   = "prompt"
    SKILL    = "skill"
    COMMAND  = "command"
    AGENT    = "agent"

KIND_FOLDER_MAP: dict[str, ContentKind] = {
    "resources": ContentKind.RESOURCE,
    "prompts":   ContentKind.PROMPT,
    "skills":    ContentKind.SKILL,
    "commands":  ContentKind.COMMAND,
    "agents":    ContentKind.AGENT,
}

@dataclass(frozen=True)
class ContentEntry:
    uri:           str
    abs_path:      Path
    kind:          ContentKind
    name:          str
    title:         Optional[str]
    description:   Optional[str]
    mime_type:     str
    size:          int
    last_modified: datetime
    front_matter:  dict = field(default_factory=dict)
```

URI Resolver

```
# src/knowledge_mcp/content/resolver.py
from pathlib import Path
from urllib.parse import urlparse, unquote

class URIResolver:
    def __init__(self, knowledge_root: Path) -> None:
        self._root = knowledge_root.resolve()

    def to_path(self, uri: str) -> Path:
        """
        Validate and resolve a file:// URI to an absolute filesystem path.
        Raises KnowledgeMCPError(-32602) on prefix violation or traversal.
        """
        if not uri.startswith("file:///knowledge/"):
            raise KnowledgeMCPError(code=-32602,
                message=f"URI must begin with file:///knowledge/. Got: {uri}")
        decoded   = unquote(urlparse(uri).path)
        candidate = (self._root / decoded.lstrip("/")).resolve()
        if not str(candidate).startswith(str(self._root)):
            raise KnowledgeMCPError(code=-32602, message="Path traversal detected")
        return candidate

    def to_uri(self, path: Path) -> str:
        return f"file:///knowledge/{path.relative_to(self._root)}"
```

Template Renderer

```
# src/knowledge_mcp/content/renderer.py
import re
from typing import Any

TEMPLATE_PATTERN = re.compile(r"\{\{(\w+)\}\}")

def render_template(template: str, arguments: dict[str, Any]) -> str:
    """
    Substitute {{argument_name}} references. Unresolved references are
    left in place. Raises KeyError if a required argument is missing
    (caller surfaces as -32602).
    """
    def substitute(match: re.Match) -> str:
        key = match.group(1)
        return str(arguments[key]) if key in arguments else match.group(0)
    return TEMPLATE_PATTERN.sub(substitute, template)
```

Filesystem Watcher

```
# src/knowledge_mcp/registry/watcher.py
import asyncio
from pathlib import Path
from watchfiles import awatch

async def watch_and_notify(
    knowledge_root: Path,
    registry,
    notify_list_changed: callable,
    notify_resource_updated: callable,
) -> None:
    async for changes in awatch(knowledge_root):
        for _, changed_path in changes:
            path = Path(changed_path)
            registry.reindex(path)
            await notify_resource_updated(registry.to_uri(path))
        await notify_list_changed()
```

Docker Deployment

```
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir ".[server]"
COPY src/ src/
VOLUME ["/knowledge"]
EXPOSE 8080
ENV KNOWLEDGE_MCP_KNOWLEDGE_ROOT=/knowledge
CMD ["uvicorn", "knowledge_mcp.server:app", "--host", "0.0.0.0", "--port", "8080"]
```

```
# docker-compose.yml
services:
  knowledge-mcp:
    build: .
    ports:
      - "8080:8080"
    volumes:
      - type: bind
        source: ./knowledge
        target: /knowledge
        read_only: true
    environment:
      KNOWLEDGE_MCP_OAUTH_ISSUER:   "${OAUTH_ISSUER}"
      KNOWLEDGE_MCP_OAUTH_AUDIENCE: "https://knowledge-mcp.maoperatingsystem.com"
      KNOWLEDGE_MCP_OAUTH_JWKS_URI: "${OAUTH_JWKS_URI}"
    restart: unless-stopped
    healthcheck:
      test:     ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout:  10s
      retries:  3
```

Health endpoint at `GET /health` returns:

```
{ "status": "ok", "version": "1.0.0", "indexed_entries": 247, "last_scan":
"2026-06-16T09:00:00Z" }
```

05 — Client Integration Guide

Product: MAOS Knowledge MCP Server
Version: 1.0
Date: 2026-06-16

Connection

Server Manifest

```
GET https://knowledge-mcp.maoperatingsystem.com/.well-known/mcp-server
```

```
{
  "mcp_version": "2025-06-18",
  "name":        "MAOS Knowledge MCP Server",
  "endpoint":    "https://knowledge-mcp.maoperatingsystem.com/mcp",
  "transport":   "http",
  "auth": {
    "required":     true,
    "methods":      ["oauth2"],
    "metadata_url": "https://auth.maoperatingsystem.com/.well-known/oauth-authorization-server"
  },
  "capabilities": ["tools", "resources", "prompts"]
}
```

Python SDK Initialisation

```
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

async with streamablehttp_client(
    "https://knowledge-mcp.maoperatingsystem.com/mcp",
    headers={"Authorization": f"Bearer {access_token}"}
) as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()
        # session ready
```

Claude Code CLI Configuration

```
// .claude/settings.json
{
  "mcpServers": {
    "maos-knowledge": {
      "transport": "http",
      "url":       "https://knowledge-mcp.maoperatingsystem.com/mcp",
      "headers": {
        "Authorization": "Bearer ${MAOS_KNOWLEDGE_MCP_TOKEN}"
      }
    }
  }
}
```

Common Patterns

Discover all skills for an application

```
result = await session.call_tool("get_skill", {
    "skill_name": "file:///knowledge/maos/dda/data-design-authority/skills/"
})
# result.structuredContent["entries"] — list of skill directories
```

Get a skill with rendered prompt

```
result = await session.call_tool("get_skill", {
    "skill_name": "gmail-triage",
    "app_uri":    "file:///knowledge/maos/dda/data-design-authority/skills/",
    "arguments":  { "max_threads": "30", "lookback_hours": "48" }
})
rendered_prompt = result.structuredContent["rendered_prompt"]
triggers        = result.structuredContent["triggers"]
dependencies    = result.structuredContent["dependencies"]
# Inject rendered_prompt into current conversation context
```

Load an agent before instantiation

```
result = await session.call_tool("load_agent", {
    "agent_name": "dda-analyst",
    "app_uri":    "file:///knowledge/maos/dda/data-design-authority/agents/"
})
agent         = result.structuredContent
system_prompt = agent["system_prompt"]
model         = agent["model"]
tools_allowed = agent["tools_allowed"]
memory_config = agent["memory"]
```

Search then retrieve

```
search = await session.call_tool("search_knowledge", {
    "query":         "data retention",
    "folder_uri":    "file:///knowledge/maos/dda/data-design-authority/",
    "content_types": ["resource", "prompt"],
    "max_results":   5
})
top_uri = search.structuredContent["results"][0]["uri"]
content = await session.call_tool("get_resource", { "uri": top_uri })
```

Get relevant sections of a large document

```
result = await session.call_tool("get_resource", {
    "uri":   "file:///knowledge/maos/dda/data-design-authority/resources/data-model.md",
    "query": "data retention policy minimum years",
    "top_k": 3
})
chunks = result.structuredContent["chunks"]
# chunks: [{ chunk_id, section_heading, text, similarity_score }]
```

Render a prompt template

```
result = await session.call_tool("get_prompt", {
    "prompt_name": "file:///knowledge/maos/dda/data-design-authority/prompts/maturity-
assessment.prompt.md",
    "arguments":   { "org_name": "Acme Corp", "scope": "Data Governance" }
})
messages = result.structuredContent["messages"]
# Pass to LLM API
```

Subscribe to resource changes

```
await session.subscribe_resource(
    "file:///knowledge/maos/dda/data-design-authority/agents/dda-analyst.agent.md"
)
# Re-fetch and reload agent definition on notifications/resources/updated
```

Per-Consumer Guidance

DDA AI Chat

At session start: call `load_agent` with `dda-analyst` to retrieve system prompt and tool config; subscribe to the agent file URI; call `list_prompts` scoped to the prompts folder to populate the prompt picker.

During session: use `get_prompt` to render prompt templates; `search_knowledge` for knowledge lookup; `get_skill` when the user triggers a skill by phrase match against `triggers[]` — `get_skill` returns the rendered prompt and typed metadata in one call.

AI Agile Pipeline Orchestrator

At pipeline start: call `load_agent` with `pipeline-orchestrator`; call `get_skill` for each skill in `agent.skills[]` to pre-cache SKILL.md content; subscribe to skill file URIs for invalidation.

During execution: use `get_command` with arguments supplied — the `resolved_command` field in `structuredContent` returns the command string with arguments substituted, ready to pass to the target MCP server; use `get_resource` to retrieve reference schemas, supplying `query` to retrieve only relevant sections from large documents.

Claude Code CLI Sessions

Once connected via the settings above, Claude Code can call any tool directly in a session:

```
> Use get_skill to load the data-lineage-review skill for this task
> Search for resources related to the concept-type enum before making changes
> Load the dda-analyst agent definition and use its tool allowlist as a constraint
```

Error Handling

Code	Client Action
`-32002` Resource not found	Do not retry without correcting the URI
`-32602` Invalid params	Fix the URI or argument before retrying
`-32603` Internal error	Retry with exponential backoff; alert on repeated failure
HTTP `429` Rate limited	Honour `Retry-After`; implement client-side request queuing
HTTP `401` Unauthorised	Refresh token via OAuth 2.1 refresh flow; retry once

Offline / Unavailable Behaviour

Cache the last successful response per URI with a 5-minute TTL. On connection failure or `5xx`, serve the cached version and log a warning. On cache miss with server unavailable, surface a user-visible notice and continue operating — do not surface server unavailability as an agent failure.

Roadmap

v1.1 — Operational hardening

- Prometheus-compatible `/metrics` — request counts, latency, index size, active SSE connections
- Structured JSON logging with correlation IDs, client ID, URI, latency per request
- Content validation CLI: `knowledge-mcp validate ./knowledge/` — checks front-matter schema compliance for all typed files
- BM25 weight tuning for FTS5 based on observed query patterns

v1.2 — Discovery enhancements

- Tag-based filtering in `search_knowledge` — tags declared in front-matter
- `triggers` index — `search_knowledge` searches skill and command `triggers[]` arrays for phrase-match discovery
- `list_applications` tool — lists all application nodes with entry counts per content type
- Dependency graph resolution — `get_skill` and `load_agent` optionally resolve dependency chains

v2.0 — Write path via GitHub PR *(conditional on confirmed requirement)*

- `propose_resource` tool — agent proposes new content; server opens a GitHub PR via the GitHub MCP server
- Staging sub-tree — `/staging/` accessible to proposing agent; promoted to `/knowledge/` on merge

ROADMAP — MAOS Knowledge MCP Server

Date: 2026-06-16

This roadmap describes the evolution from a simple embedded knowledge folder co-located with a single application, through to a centralised governed knowledge platform serving the full MAOS suite. Each version is independently deployable and leaves the API surface of prior versions intact.

---

v1.0 — Embedded Knowledge: Resources and Skills

The knowledge folder lives inside the application repo and is deployed with it. The MCP server is co-located with the application — it is not a shared service. No cross-application sharing. No dedicated knowledge infrastructure. The pattern is proven at zero operational cost before any architectural commitments are made.

Scope

Two content types only: **resources** and **skills**. Everything else is deferred until the pattern is validated.

Folder Structure

```
{application-repo}/
├── knowledge/
│   ├── resources/          # reference files — markdown, JSON, CSV
│   │   └── glossary.json
│   └── skills/             # skill packages
│       └── gmail-triage/
│           ├── SKILL.md
│           └── parse_state.py
├── src/
│   └── ...
└── docker-compose.yml      # knowledge-mcp co-located with the application
```

The knowledge directory is mounted read-only into the MCP server container. It is versioned and deployed as part of the application — a knowledge change is a code change, goes through the same PR and CI process.

Tool Surface

Tool	Purpose
`get_resource`	Retrieve a file or directory listing by URI
`search_knowledge`	FTS5 keyword search across resources and skills
`search_resources`	Shortcut — search_knowledge scoped to resources

Tool	Purpose
`search_skills`	Shortcut — search_knowledge scoped to skills, searches triggers[]
`get_skill`	Retrieve SKILL.md metadata and rendered prompt
`get_prompt`	Render a .prompt.md file with argument substitution
`list_prompts`	Enumerate all promptable content (skills and any .prompt.md files)

MCP Primitives

Primitive	Methods
Resources	`resources/list`, `resources/templates/list`, `resources/read`, `resources/subscribe`
Prompts	`prompts/list`, `prompts/get`
Tools	`tools/list`, `tools/call`

Infrastructure

- Python 3.12 + FastMCP
- SQLite FTS5 — embedded, zero external dependency
- `watchfiles` for filesystem change detection
- Docker, knowledge directory as read-only bind mount from application repo
- OAuth 2.1 + PKCE

URI Convention

```
file:///knowledge/resources/{+path}
file:///knowledge/skills/{skill-name}/SKILL.md
```

The root is `file:///knowledge/` — simple, flat, no domain/app segments needed at this stage. The URI convention is forward-compatible; the domain/app segments are added in v2.0 without breaking existing URIs for single-application deployments that choose to adopt them.

What Is Out of Scope

Prompts as a standalone content type, commands, agents, cross-application sharing, entitlements, semantic search, and authoring tooling are all deferred. The goal of v1.0 is a working, deployable server that a single application can use to serve its own knowledge content to its own agents.

---

v1.1 — Full Content Types, Still Embedded

Adds the remaining three content types within the same embedded, co-located deployment model. The knowledge folder structure gains `prompts/`, `commands/`, and `agents/` folders alongside the existing `resources/` and `skills/`. No new infrastructure. The MCP server is still part of the application repo.

What Changes

New content types:

Type	Folder	Entry point
Prompt	`knowledge/prompts/`	`.prompt.md` / `.prompt.json`
Command	`knowledge/commands/`	`.cmd.md` / `.cmd.json`
Agent	`knowledge/agents/`	`.agent.md` / `.agent.json`

New tools:

Tool	Purpose
`search_prompts`	Shortcut — search_knowledge scoped to prompts
`search_commands`	Shortcut — search_knowledge scoped to commands, includes danger_level in results
`search_agents`	Shortcut — search_knowledge scoped to agents, includes model and tools_allowed in results
`get_command`	Retrieve command definition; returns resolved_command with arguments substituted
`list_agents`	Enumerate agent definitions
`load_agent`	Return full agent definition with rendered system prompt

Front-matter additions: - `tags` field added to all typed schemas — enables faceted filtering across all search tools - `triggers[]` indexed as a dedicated FTS5 column with boosted weight — phrase-match queries against `search_skills` and `search_commands` now surface matching content by natural language trigger

Operational additions: - Prometheus-compatible `/metrics` endpoint - Structured JSON logging with correlation IDs per request - `knowledge-mcp validate ./knowledge/` CLI — checks front-matter schema compliance for all typed files before deployment

What Does Not Change

Deployment model, infrastructure, URI convention, and all v1.0 tools are unchanged.

---

v2.0 — Centralised Knowledge Server

The knowledge folder moves out of the application repo into a dedicated Git repository. The MCP server becomes a shared service running independently of any application. Multiple applications each have their own sub-tree. Applications change from running their own embedded server to connecting as MCP clients to the shared server.

Architectural Change

```
Before (v1.x):
  application-repo/
  ├── knowledge/          # owned by this application
  └── src/

After (v2.0):
  knowledge-repo/         # standalone Git repo
  └── knowledge/
      └── {domain}/
          └── {sub-domain}/
              └── {application}/
                  ├── resources/
                  ├── skills/
                  ├── prompts/
                  ├── commands/
                  └── agents/

  application-repo/       # no longer contains knowledge/
  └── src/                # connects to shared knowledge MCP server as a client
```

URI Convention Update

The URI root gains domain and application segments to namespace content across multiple consumers:

```
file:///knowledge/{domain}/{sub-domain}/{application}/resources/{+path}
file:///knowledge/{domain}/{sub-domain}/{application}/skills/{skill}/SKILL.md
```

Applications that adopted the flat v1.x URI convention update their client configuration to use the new root. All tool names and parameter schemas are unchanged.

New Tools

Tool	Purpose
`list_applications`	Enumerate all application nodes in the knowledge directory with per-type entry counts

New Infrastructure

- Dedicated knowledge Git repository with CD pipeline syncing main branch to the server's mounted filesystem

- `watchfiles` now serves multiple consumers — `listChanged` and `updated` notifications fan out to all subscribed clients
- Shared OAuth 2.1 authorisation server; per-application scopes available but flat `knowledge:read` remains the default

Migration Path

1. Create `knowledge-repo` and move each application's `knowledge/` folder to its namespaced sub-tree
2. Deploy the centralised knowledge MCP server pointing at the new repo mount
3. Update each application's MCP client configuration to point at the shared server endpoint and the new URI root
4. Remove the embedded server from each application's `docker-compose.yml`

No tool API changes. No knowledge content changes beyond the folder relocation.

---

v2.1 — Search, RAG, and Vector Retrieval

This version adds pgvector-backed semantic search and chunk retrieval. Nothing in the v1.x or v2.0 surface changes — v2.1 is purely additive.

New Capability: `search`

Replaces `search_knowledge` as the standard discovery tool. Combines FTS5 keyword ranking and pgvector semantic ranking using Reciprocal Rank Fusion. Accepts identical parameters to `search_knowledge` — all typed shortcuts (`search_skills` , `search_agents` etc.) delegate to `search` automatically once v2.1 is deployed. `search_knowledge` remains available as a fallback.

```
Input:  query, folder_uri?, content_types?, tags?, top_k
Output: [{ uri, name, title, content_type, snippet, rrf_score, keyword_rank, semantic_rank }]
```

New Capability: Chunk Retrieval via `get_resource`

Chunk retrieval is not a new tool — it is an extension of `get_resource` activated by supplying an optional `query` parameter. When `query` is present, the server returns ranked sections of the document rather than the full content.

```
get_resource(uri, query="data retention policy", top_k=3)
→ { chunks: [{ chunk_id, section_heading, text, similarity_score }] }
```

Individual chunks are addressable via fragment URI:

```
file:///knowledge/maos/dda/data-design-authority/resources/data-model.md#chunk=3
```

New Infrastructure

Component	Purpose
Supabase pgvector	`knowledge_vectors` schema — one embedding row per file, one per chunk
`registry/embeddings.py`	Generates embeddings on reindex, upserts to pgvector with `embedding_model` and `embedding_dimensions` stored as row metadata
`registry/chunker.py`	Section-based splitting for markdown (on `##` headings); sliding window with overlap for prose
`tools/search_tools_v2.py`	Implements `search` ; `get_resource` chunk path handled in `resource_tools.py`

Graceful Degradation

`search` and chunk retrieval via `get_resource` are gated by `KNOWLEDGE_MCP_SEARCH_ENABLED=true`. When the flag is absent or false both return `-32603` with message `"Not available — pgvector not configured"`. All FTS5 tools remain fully operational.

Open Decision

OD-005 — Embedding model versioning: Store `embedding_model` and `embedding_dimensions` as metadata on every pgvector row. A model change without re-indexing produces silently degraded results — the stored metadata enables the server to detect and reject stale embeddings after an upgrade.

---

v3.0 — Knowledge Authoring and Entitlements

This version transforms the server from read-only infrastructure into a governed knowledge platform. It is the first version with a write path.

Knowledge Authoring Frontend

A web application for creating and editing knowledge content without touching Git or the filesystem directly. Targeted at practitioners — CDOs, data stewards, solution architects — who need to publish prompts, skills, and agent definitions but are not developers.

Capabilities:

- Create and edit all five content types through structured forms — front-matter fields rendered as typed inputs, body rendered as a markdown editor
- Front-matter schema validation on save — malformed content is rejected before it reaches the server index

- Preview rendering — prompt templates rendered with sample arguments, skills displayed as their final injected form
- Draft and publish states — content exists in draft (visible only to the author) until explicitly published to the live index
- Publish workflow — optional approval gate; a reviewer approves or rejects before content goes live
- Version history — each published version is a Git commit; authors can view diff and roll back

Implementation approach: The authoring frontend calls a write API layer that manages Git operations on the knowledge repository. The MCP server itself remains read-only — it continues to serve from the filesystem as before. The write path is entirely in the authoring layer, not in the MCP server.

Entitlements

Per-application access control replacing the flat `knowledge:read` scope.

Read entitlements: A client's access token carries claims scoping it to one or more application sub-trees. A token scoped to `maos/dda/data-design-authority` cannot read content from `maos/pipelines/ai-agile`. The MCP server enforces this at the URI validation step — a request for a URI outside the token's scoped sub-trees returns `403`.

Author entitlements: The authoring frontend enforces who can create and edit content within each application sub-tree. Separate from read entitlements — a user may be able to read across multiple applications but author only within their own.

Approval entitlements: Designated reviewers per application sub-tree. The publish workflow routes approval requests to the correct reviewer based on the content's application path.

What Does Not Change

The MCP server tool surface, URI convention, primitive methods, and all consumer integrations are unchanged. Entitlements are enforced at the token and URI validation layer — the tools themselves have no awareness of entitlements.

Summary

Version	Deployment Model	Content Types	Search	Authoring
v1.0	Embedded in app repo	Resources, Skills	FTS5 keyword	Git / filesystem
v1.1	Embedded in app repo	+ Prompts, Commands, Agents	FTS5 + tags + triggers	Git / filesystem

Version	Deployment Model	Content Types	Search	Authoring
v2.0	Centralised shared server	All five	FTS5 + tags + triggers	Git / filesystem
v2.1	Centralised shared server	All five	+ Hybrid (FTS5 + pgvector + RRF), chunk retrieval	Git / filesystem
v3.0	Centralised shared server	All five	Hybrid + chunk retrieval	Web authoring frontend + entitlements

---

Discovery and Retrieval Architecture

The search/get boundary is stable across all versions. Search tools are discovery — they return URIs and snippets. Get tools are retrieval — they return content at a known URI. v2.1 adds richer discovery; v3.0 adds authoring. The boundary itself never changes.